

Components Architecture Constitution

1. Vision

Components is a headless platform for electronic design, simulation, validation, learning, automation, and AI-assisted construction.

Components is not tied to any particular graphical interface, operating system, programming language, AI provider, or storage provider.

A user may interact with Components through:

- Command-line tools
- APIs
- AI agents
- Voice interfaces
- Python SDKs
- Web clients
- Stand-alone applications
- Tablet-first Boards
- IDE extensions
- Third-party clients

All clients operate on the same package model and receive the same validation and simulation results.

2. Core Principle

Components owns the truth. Boards own the experience.

Components Platform is authoritative for:

- Component definitions
- Circuit connectivity
- Package structure
- Pin resolution
- Dependency resolution
- Validation
- Simulation
- Export
- History
- Transactions
- Revisions

A Board is a replaceable client that presents and edits information through Components Platform.

No Board may define independent circuit semantics that differ from Components Platform.

3. User-Facing Model

A user works with three primary concepts:

3.1 Component

Component describes what the circuit is.

It defines:

- Inputs and outputs
- Chips and modules
- Connections
- Nets
- Buses
- Interfaces
- Hierarchy
- Functional relationships

Component does not define positions, routes, colors, zoom levels, or visual layout.

3.2 Board

Board describes how a Component is presented and interacted with.

It may define:

- Placement
- Grouping
- Alignment
- Routing
- Bus presentation
- Labels
- Portals
- Visibility
- Pin display
- Learning views
- Engineering views
- Breadboard views
- Timing views
- Debug views

A Board may not change electrical connectivity.

3.3 Resources

Resources describe what a package depends on and carries with it.

Resources may include:

- Component libraries
- Submodules
- Visual themes
- Custom symbols
- DIP package drawings
- Datasheets
- Lessons
- Images
- Audio
- Tests
- Examples
- Documentation
- Export profiles

Resources may affect availability and presentation, but visual resources may not redefine electrical truth.

4. Package

A Package is the portable document boundary of Components.

A Package contains or references:

- Component source
- One or more Boards
- Resources and dependencies
- Package metadata
- Tests
- Optional bundled modules
- Optional custom visuals
- Revision information

The user-facing package may appear as:

```
MyProject/  
├─ main.component  
├─ boards/  
│   ├─ learning.board  
│   └─ engineering.board
```

```
| └─ breadboard.board
└─ components.toml
```

The platform may internally maintain:

- Canonical IR
- Dependency lock data
- Simulation caches
- Validation indexes
- Generated Verilog
- Generated SystemVerilog
- Revision history
- Render models

These internal artifacts are platform-managed and are not primary user sources.

5. Component Language

Component is a concise, declarative, structural hardware language.

Its primary purpose is to express relationships rather than execution steps.

Example:

```
component CounterDemo {
  input Clock
  input Reset

  chip Counter : 74HC161
  chip LED[3:0] : LED

  connect Clock -> Counter.CLK
  connect Reset -> Counter.CLR_N
  connect Counter.Q[3:0] -> LED[3:0].A
}
```

Component should be:

- Student-friendly
- AI-friendly
- Voice-friendly
- Deterministic
- Human-readable
- Git-friendly

- Structurally mappable to visual blocks
- Exportable to SystemVerilog and Verilog

The language should borrow selected structural ideas from SystemVerilog, including:

- Modules
- Typed ports
- Packed vectors
- Named connections
- Interfaces
- Roles similar to modports
- Hierarchical instances

The initial language should avoid unnecessary procedural and verification features.

6. Physical Pin Identity

Every physical pin has both semantic and physical identities.

Example:

```
Counter.CLK  
Counter.CLK@2  
Counter.pin2
```

These references may resolve to the same physical endpoint.

The canonical model should retain:

- Instance identity
- Logical port name
- Physical package
- Physical pin number
- Direction
- Electrical behavior
- Aliases

Logical identity is primary:

```
Counter.CLK
```

Physical binding is package-specific:

DIP16 pin 2

User-facing engineering displays should normally show both:

CLK (Pin 2)

7. Board Model

Board is a presentation model, not a circuit model.

Example:

```
board CounterLearning {  
    place Counter center  
    stack LED[3:0] vertical right-of Counter  
  
    bundle Counter.Q[3:0] as COUNT  
    route COUNT -> LED[3:0] fanout  
  
    show pin names  
    show pin numbers  
    hide power wires  
}
```

A Component may have multiple Boards.

Examples:

```
learning.board  
engineering.board  
breadboard.board  
timing.board  
debug.board  
presentation.board
```

All Boards reference the same Component design.

8. Headless Platform

Components Platform must function without a graphical interface.

Required access methods may include:

```
CLI
Local API
Remote API
MCP
Python SDK
JSON operations
Package input/output
```

Example CLI:

```
components check counter-demo.cpkg
components simulate counter-demo.cpkg
components export counter-demo.cpkg --to systemverilog
components inspect counter-demo.cpkg
```

A visual Board is only one client among many.

9. Operation Protocol

Every edit from mouse, pen, voice, text, AI, CLI, or Python must resolve into a structured operation.

Example:

```
{
  "domain": "component",
  "action": "connect",
  "source": "Counter.Q0",
  "target": "LED0.A"
}
```

Board operation:

```
{
  "domain": "board",
  "action": "place",
```

```
"target": "LED0",  
"relative_to": "Counter",  
"position": "right"  
}
```

Resource operation:

```
{  
  "domain": "resources",  
  "action": "add_dependency",  
  "target": "74hc161",  
  "version": "^1.0"  
}
```

The protocol must support:

- Validation
- Transactions
- Undo
- Redo
- Replay
- Revision control
- Collaboration
- AI tool calling
- Offline operation
- Conflict detection

10. JSON and Python

JSON operations are the canonical command protocol.

Python is an automation and scripting client.

Python must call the same public operation service as every other client.

Allowed:

```
project.component.connect(  
    "Counter.Q0",  
    "LED0.A",  
)
```

Not allowed:

```
project._graph.nodes["Counter"]["pins"][0] = ...
```

Python must not bypass:

- Validation
- History
- Permissions
- Revisions
- Undo and redo
- Package rules

11. Board Clients

A Board is a presentation and interaction client for Components.

Possible Board implementations include:

- Web Board
- Tablet Board
- Desktop Board
- VS Code Board
- Jupyter Board
- Read-only Board
- Teaching Board
- Debug Board
- Breadboard Board
- Terminal Board
- AR or VR Board

Boards may be web-based, native, embedded, or third-party.

No Board owns the package.

12. Tablet-First Experience

The primary visual Board should be designed tablet-first.

Desktop support should expand the workspace rather than replace the tablet interaction model.

Primary interaction methods include:

- Touch
- Pen

- Mouse
- Keyboard
- Voice
- AI assistance

Core principles:

- Content first
- Minimal chrome
- Direct manipulation
- Few permanent tools
- Searchable commands
- No ribbon interface
- No deeply nested menus
- Multiple Boards may be opened simultaneously
- Multiple windows and screens must be supported

A desktop environment may display:

Screen 1: Learning Board
Screen 2: Breadboard Board
Screen 3: Component source or simulation
Tablet: Inspector or command interface

13. AI Router

AI is not embedded as a mandatory dependency of Components Platform.

An AI Router is an independent client or provider layer.

It translates:

Natural language
Voice
High-level intent

into:

Components operations
Queries
Validation requests

Simulation requests
Explanations

AI providers may be:

- Local
- Cloud-based
- School-hosted
- User-selected
- Offline
- Commercial
- Open source

Components Platform must not depend on a specific AI provider.

14. AI Safety and Control

AI actions should support three modes:

Inspect

Read and explain without changing the package.

Suggest

Return a proposed transaction or patch without applying it.

Apply

Apply an approved transaction.

AI-generated changes must be:

- Structured
- Validated
- Attributed
- Reversible
- Recorded in history
- Reviewable

AI should never directly edit internal platform files.

15. Team and Collaboration

Team support should be designed architecturally before implementation.

The initial implementation may remain single-user.

Every operation should still contain:

- Operation ID
- Session ID
- Author
- Agent
- Timestamp
- Base revision

A session may represent:

- A human user
- An AI agent
- A script
- A CLI process
- An auto-layout service
- A Board client

Future collaboration may add:

- Workspaces
- Roles
- Permissions
- Shared packages
- Git workflows
- Classroom workflows
- Real-time collaboration

The core platform should not require redesign when collaboration is added.

16. Export

Component designs should export independently of Board presentation.

Initial export targets:

- Structural SystemVerilog
- Verilog-2001
- Canonical JSON IR
- Package snapshot

Future targets may include:

- Synthesizable RTL
- Testbench generation
- Documentation
- SVG diagrams
- Netlists
- FPGA projects
- PCB-oriented formats

Board files must not be required for structural HDL export.

17. SystemVerilog Relationship

Component is not a replacement for SystemVerilog.

Component is a physical-component-aware, student-friendly structural language that maps cleanly to SystemVerilog.

Example mapping:

Component component	→ SystemVerilog module
Component chip	→ module instance
Component input/output	→ module port
Component bus	→ packed vector
Component connection	→ named port connection or net
Component interface	→ SystemVerilog interface
Component role	→ modport-like structure
Package pin binding	→ metadata or export annotation

Component remains simpler and more conversational than SystemVerilog.

18. Source of Truth

The sources of truth are:

Component
Boards
Resources
Package metadata

Derived artifacts are not authoritative.

Derived artifacts include:

- Generated Verilog
- Generated SystemVerilog
- Render cache
- Simulation cache
- Layout cache
- Dependency lock resolution
- Diagnostics indexes

Generated outputs may be deleted and recreated.

19. Replaceability Principle

Everything is replaceable except the Component model and Package contract.

Replaceable elements include:

- Board implementation
- Renderer
- UI framework
- Programming language
- AI provider
- Storage provider
- Simulation backend
- Export implementation
- Operating system

Stable contracts must remain independent of implementation choices.

20. Architectural Boundaries

Components Platform owns

- Semantics
- Package validation
- Pin resolution
- Connectivity
- Simulation
- Export
- Revision history
- Operation application

Board owns

- Presentation
- Interaction
- Gestures
- Local selection
- Pan and zoom
- Rendering
- Window arrangement

AI Router owns

- Natural-language interpretation
- Voice interpretation
- Intent planning
- Tool selection
- Explanation style

Storage Provider owns

- Persistence
- Synchronization
- Remote storage
- Git integration
- Cloud integration

No layer should silently duplicate another layer's responsibility.

21. Architecture Principles

1. Components is a platform, not an application.
2. Components owns the truth; Boards own the experience.
3. A Board is replaceable; a Package is portable.
4. Every client speaks through the same operation model.
5. Component describes meaning; Board describes presentation.
6. Visual resources may not redefine electrical truth.
7. AI is a client, not a core dependency.
8. Every action must be structured, validated, attributable, and reversible.
9. The platform must work offline.
10. The platform must remain useful without AI.
11. Tablet-first does not mean tablet-only.
12. Generated artifacts are never primary sources.
13. No front-end-only circuit semantics.
14. Physical pin numbers and logical pin names must remain linked.
15. Architecture and contracts come before implementation.